CSE231 Operating System (Section A), Quiz 03 Monsoon 2025
Time allocated: 4pm – 4:20pm

Name	
Roll Number	

Instructions:

- This **is a closed book and closed notes quiz. Please be aware of strict plagiarism policy.**
- For questions requiring justification, please be as concise as possible. 2-3 sentences would be the ideal size of a justification. No extra pages will be provided.
- **NO EXTRA ANSWER SHEET WILL BE PROVIDED. USE THE GIVEN SPACE WISELY**

Question-1: Is there a possibility that "void ***P** = malloc(10)" in two different processes running simultaneously in an OS return the **same** pointer "**P**"? Briefly justify? **[2 marks]**

Answer: Yes [+1 marks], as P points to a virtual address and each process has its own isolated virtual address space. [+1 marks]

Question-2: Using pseudocode show the translation of an address "**P**" using the base and bound approach. Assume **B** represents base and **L** represents bounds. You code must use these three variables. **[2 marks]**

Answer:

```
If (P >= L) throw exception (or printf, or a comment, etc);      [+1 marks]
Else {                                                           [+0.25 marks]
  Physical address = Base + P.                                   [+0.75 marks]
}
```

Question-3: Which one of the following scheduling policies may cause starvation: FCFS, MLFQ, and RR? Brief justification. **[2 marks]**

Answer:
MLFQ [+1 marks]. A continuous stream of newly added processes would get a chance to execute at high priority queue, leading to starvation of the processes in lower priority queues. [+1 marks]

Note that starvation does not mean a process never gets a chance to execute, but it means the process remains blocked for unexpected time. If a process never gets a chance to execute then it means its deadlocked. Promotion of processes happens from low priority queue to higher priority queue, but it is not easy to find optimal duration for promotion, especially when new jobs keep coming and are being added to the higher priority queue.

Question-4: Considering a large and identical set of ready jobs of CPU-burst type, each with different execution times, which of the FCFS, MLFQ and RR scheduling policies would result in the **least** context-switch overhead? Brief justification. **[2 marks]**

Answer:
FCFS [+1 marks]. It is non-preemptive and a process will not context switch once admitted to the running queue. [+1 marks]

Question-5: Between an implicit list allocator and a buddy allocator, which one is more susceptible to external fragmentation? Briefly justify. **[2 marks]**

Answer:

Implicit list [+1 marks], as unlike buddy allocator it uses arbitrary block sizes for allocations leading to scattered free space over time. [+1 marks]